

Tiny Federated Learning with Blockchain for Privacy and Security Preservation of MCU-Based IoT Applications

G. P. Rimoli, B. Boi, P. Fusco, C. Esposito, and M. Ficco

Department of Computer Science of University of Salerno

Via Giovanni Paolo II, 132 - 84084 Fisciano (SA) - Italy

{grimoli, bboi, pfusco, esposito, mficco}@unisa.it

Abstract—In several Internet of Things (IoT) application contexts, such as autonomous vehicles, healthcare, and smart cities, massive amounts of data are produced at the edge and used in neural networks deployed in central servers or the cloud. On the other hand, physical or legal constraints may restrict the use of this data only locally. Thus, the development of secure and efficient traditional Machine Learning solutions in the IoT context can be a huge challenge. Therefore, this paper combines an approach based on Tiny Federated Learning and Transfer Learning with on-board training, as an effective paradigm to continuously analyze data locally without having to transfer sensitive data to untrusted servers and networks. Moreover, a decentralized blockchain-based federated learning framework is implemented to provide tamper-proof data protection and resistance to malicious or compromised tiny devices. A prototype is created based on the Hyperledger Fabric and real resource-constrained microcontrollers to assess the viability of the proposed solution.

Index Terms—TinyML, on-board training, Federated Learning, Blockchain, MCU.

I. INTRODUCTION

The proliferation of resource-constrained Internet of Things (IoT) smart devices in various real-life contexts, such as healthcare, smart homes, and industrial automation, has led to the development of numerous applications that involve the direct or indirect use of large volumes of data. Due to the sheer volume of this data, it is typically analyzed using Artificial Intelligence models running on servers or cloud systems. However, the sensitivity of the processed data or the latency times required to manage efficient responses require that its use be restricted to the local nodes where the data is generated [1].

In this work, we propose the combination of two well-known technologies adapted to the context of resource-constrained devices such as IoT devices. The first step involves integrating on-board training with Federated Learning (FL).

We developed a specialized C library to facilitate online TinyML training on popular microcontrollers (MCUs) such as Arduino, Espressif, and STM boards. Then, we implement data privacy through FL technology, ensuring that data analysis occurs locally and in a decentralized manner, thus avoiding the need to transfer sensitive information to a central server or cloud service.

The second step consists of integrating blockchain technology in the context of the FL system. We establish a secure, decentralized and transparent logging system that not only enhances the security and auditability of the models, but also supports the recovery of the system's integrity post-compromise. Specifically, as we will illustrate in the subsequent section, blockchain serves as a facilitator to realize a robustly available and secure historical database, using its decentralized and secure characteristics [6]. When the overall accuracy of the global model consistently declines within a defined time frame, presumably due to an attack, the central node restores the global model to a state computed before the onset of the accuracy decline.

This integration is important because an attacker may compromise or replace a subset of local nodes to lead the global model to misclassifications or reduce its accuracy. For example, Chen et al. [3] presented a model poisoning attack, in which the weights generated by local nodes were manipulated to compromise the global model. In the Federated Learning architecture, the central node does not have access to the local data and cannot detect well-crafted malicious models from benign ones. If a large number of local nodes are attacked or replaced, malicious nodes can quickly take over the legitimate ones and compromise the overall federated learning solution. To reduce the impact of colluded malicious nodes, the central node can randomly select the nodes to collect the weights among the available ones, but this approach may not resolve the issue if the number of malicious local nodes is overwhelming [4]. More simply, attackers can also inject locally manipulated training data into local nodes, implementing a data poisoning attack used to compromise local on-board training, causing the global model to diverge from the correct one [5].

The current literature suggests protecting against the previously described attacks by making advanced aggregation at the central node to remove the possible outliers caused by the compromised local nodes, or by exploiting crypto-solutions to protect the communications. Another solution may consist of leveraging trust management to infer whether a local node is compromised and to exclude those with the lowest credibility. On the one hand, the first kind of solution is vulnerable to non-IID datasets (Non-Identically Distributed), which may

cause slower convergence to global models as they cause the exclusion of legitimate nodes. On the other hand, crypto-solutions can only cope with external attacks and may not be implemented in resource-constrained MCU devices. Moreover, both solutions are ineffective in protecting against cases in which the attacker can physically capture a local node.

The rest of the work is structured as follows: Sec. II describes the current state of the art of the most commonly used approaches related to on-board TinyML and Blockchain-based Federated Learning. Sec. III presents the on-board training implementation on common MCUs. The Tiny Federated Learning solution is described in Sec. IV. Sec. V presents the Blockchain-Based solution. Conclusions and future works are presented in Sec. VI.

II. RELATED WORK

Although several works in the literature faced Federated Learning and Transfer Learning in IoT ecosystems, they do not investigate aspects related to on-line training in severe resources-constrained tiny development boards [9], [10]. Specifically, the proposed TinyML solutions either supported only the batch/offline pre-training model (using a large amount of pre-collected data on a powerful server), or implemented Federated Learning solutions using most embedded-Linux devices like Raspberry Pi in the edge network [11], [16]. For example, modern deep training frameworks, such as TensorFlow Lite Micro (TFLM), X-CUBE-AI from STM, and PyTorch Mobile, can be used to perform offline training. The output of such libraries is then miniaturized for fitting into the small memory of MCUs through transformations. This approach is widely used in various fields for detection, classification, and prediction problems [12], [13]. On the other hand, this approach requires that sensitive data, or part of it, is transferred to central servers for the network model fine-training.

Recent works have discussed how to learn the newest knowledge from on-field data on tiny devices [14], [15], [16], [17]. Young *et al.* [24] proposed an on-device model training implementation, reducing the computation cost of the backpropagation and optimizing the memory consumption. However, the presented solution is only implemented on embedded devices such as Raspberry Pi. Ren *et al.* [16] represents the first solution that enables incremental on-device training on streaming data. In [17], a solution that combines on-board training and Federated Learning on resource-constrained tiny devices is proposed. Moreover, an accurate experimental analysis in terms of accuracy, power consumption, training, and inference is presented.

Blockchain is used to guarantee the consistency of replicated data in a distributed environment. Distributed consensus algorithms are used to provide data immutability from intentional and unintentional tempering thanks to the used crypto primitives. Recently, the blockchain has been used in the federated learning context to enable clients and servers to exchange and verify weights of their respective models [18], [19], [20], [21], [22], [23]. Specifically, in the context of

Federated Learning, Blockchain can be used to log the various versions of the global model and allow rolling back to the most accurate version in case of compromises of the federated model by a malign. Moreover, the blockchain is used to track history or make model updates unchangeable to implement the accountability principle of GDPR and allow a human-in-the-loop solution demanded by the upcoming EU AI regulation.

This study, in contrast to previous approaches, integrates on-board TinyML Training with Federated Learning supported by Blockchain technology to reinforce the security and privacy of MCU-based IoT applications. The implementation of this defined architecture has the potential to improve security while considering the inherent limitations of IoT nodes.

III. TINYML WITH ON-BOARD TRAINING

The considered TinyML architecture consists of heterogeneous resource-constrained devices, which perform on-board inference to reduce latency in decision-making processes and increase privacy. Moreover, to avoid transferring sensitive data to powerful machines or the cloud for training a centralized ML model, on-board training has been implemented. A C library has been developed to ensure compatibility with popular microcontrollers produced by Arduino, STMicroelectronics, and Espressif [17]. Specifically, it is an extension of Genann, which is a lightweight neural network library written in pure C without dependencies, used to implement backpropagation training, and supports linear and sigmoid activation functions.

The neural network deployed on each tiny device has been configured with four layers. In particular, some works in the literature have suggested the use of 50 neurons for each hidden layer. On the other hand, such a configuration would require excessive memory consumption for mostly microcontrollers. Therefore, it has been configured with an input layer with 187 neurons, two hidden layers, each containing 25 neurons, and an output layer with 5 neurons.

To test the implemented tiny neural network, the ECG heartbeat dataset has been used, which is a collection of heartbeat signals, *i.e.*, electrocardiogram shapes of heartbeats in normal conditions of healthy, myocardial infarction, and arrhythmia. The neural network is used to predict a condition of myocardial infarction or arrhythmia. The dataset contains 109,446 samples, with 187 input features (*i.e.*, ECG input channel) and 5 output classes. Considering the resource constraint of the used devices, the dataset is not entirely used. In particular, 8000 samples are used for training and 10000 samples for testing.

Tab. I shows the memory consumption required to execute the network on ESP32 board. Depending on the lookup table size the required memory ranges between ~22kB to ~44kB.

Tab. II shows the accuracy of the network achieved during the training and testing steps. The difference in accuracy, precision, and F1 between training and testing dataset parts is more pronounced if compared to the difference in the Multi-

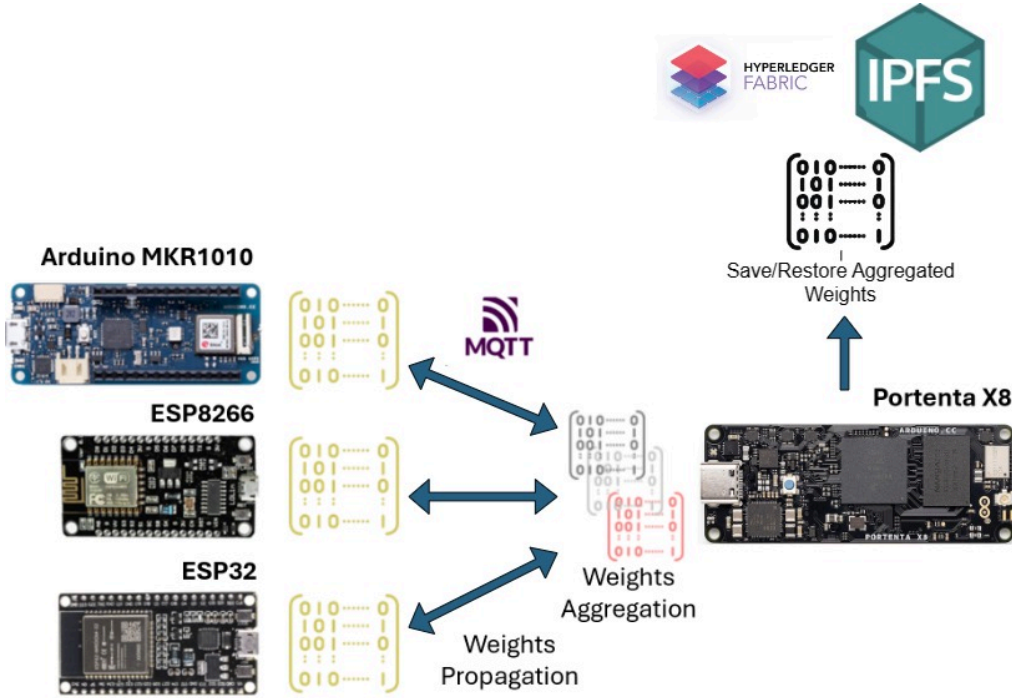


Fig. 1. Federated Learning architecture.

TABLE I
MEMORY CONSUMPTION ON ESP32 BOARD.

Characteristics	Size	Memory consumption
Neurons	5777	23.152 kB
Lookup table	1024	4.073 kB
Lookup table	4096	16.384 kB
Input float array	187	0.752 kB
Output float array	5	0.02 kB
Char array to store each dataset sample	15	0.015 kB

Class Entropy (MCE) and Mean Squared Error (MSE), where MCE is the entropy of the predicted class probabilities:

$$L(\hat{y}, y) = - \sum_{i=1}^K y_i \cdot \log[\hat{y}_i], \quad (1)$$

MSE is the average squared difference between predicted, $\hat{y}_i$, and actual, y_i , class values:

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2. \quad (2)$$

TABLE II
TRAINING AND TESTING RESULTS ON ESP32 BOARD.

Metrics	Training	Testing
Accuracy	88.03%	86.31%
Precision	83.31%	76.91%
F1	0.7237	0.6597
MSE	0.0184	0.0205
MCE	0.1146	0.1179

IV. TINY FEDERATED LEARNING

Federated Learning enables model training by a distributed IoT-Edge architecture without the need to directly share the data. Each IoT device manages and stores locally its private data and collaborates to collectively train the ML model, preserving privacy and security.

In the context of this work, we considered a *cross-device and horizontal FL*, in which each IoT device manages different local data, but with similar features. Moreover, a Transfer Learning technique is applied, in which the knowledge acquired from a pre-trained central model is transferred to the IoT devices to improve the performance of the tiny model without training it from scratch, as well as reduce the training time and energy consumption on the IoT device.

According to this technique, the central pre-training model represents the starting point, which includes general features and patterns. Then, a fine-tune on-board of the pre-trained model is performed, using local data. The fine-tuned model is shared with the central server. Finally, the server applies the FL technique to build the final model.

The FL solution is developed by a Python library, called Numpy [25]. The federated architecture is implemented by a set of MCUs, including ESP8266, ESP32, and Arduino MKR1010 as IoT nodes, and Arduino Portenta H7 as the central server. The MQTT protocol is used to exchange data between the central server and the IoT nodes.

In the first analysis, we assume that the IoT nodes have been trained on balanced datasets with the same class distribution. As Tab. III shows, the accuracy and precision score achieved

in each MCU node is similar to the one obtained in the pre-trained model used for the transfer learning phase. However, MCUs gained an MSE and MCE score about 35% lower than the pre-trained model.

Fig. 2 reports the accuracy as the number of epochs varies. It is greater than 0.84 after the 10th epoch. In general, the model is consistent across all the training epochs for each considered metric.

However, in a real-world scenario, IoT devices might process different data, *i.e.*, they are trained on unbalanced datasets. Therefore, assuming four different dataset distributions for the test, the comparison of the training results with the testing ones is shown in Tab. IV. Results show that the accuracy is approximately 10% lower.

V. BLOCKCHAIN-BASED FEDERATED LEARNING

The Federated Learning paradigm supports data privacy for each client, sharing only the weights of their local model with a central node, which feeds a global model and returns the new computed weights to the local nodes. These exchanges can be repeated a number of rounds until a predefined stop criterion is reached.

To strengthen the security of the model against possible tampering by attackers, in the context of this work, we have adopted a rollback mechanism of the computed federated model. The models computed at each round of the training process are persistently tamper-proof and stored in the history database. The database holds the combined weights of the models that meet the accuracy policy set during implementation. Accuracy is calculated as the average of the degrees sent by local nodes after testing their respective models. To maintain adequate accuracy, the central node restore the last model stored in the history database if its accuracy decreases within a given time window (see. Sec. V-C).

Blockchain technology has been adopted as the enabler to implement the history database, due to its decentralized and secure nature [6]. It is used to log the different versions of the global model, as well as roll back to the most accurate version if it is necessary. It leverages a distributed consensus algorithm to impose a weak consistency model among the replicas of a ledger deployed at central nodes distributed over the Internet. The ledger is structured in blocks. Each block contains the hash of the contained transactions, as a Merkle tree, and a hash of the previous block, where the genesis block is the only one not holding such a backward reference. Such a hashing mechanism guarantees integrity and tamper-proof of the data stored in the blockchain.

On the other hand, within the context of Federating Learning logs, the limited size of a block may not allow for storage of the overall set of weights of a global module. Therefore, IPFS is exploited to implement a distributed file system empowered with cryptography to provide security [6]. When a document is added over IPFS, it is assigned a unique Content Identifier (CID) and is subsequently sliced into many small chunks, each of which is hashed. The CID also acts as a kind of "proof"

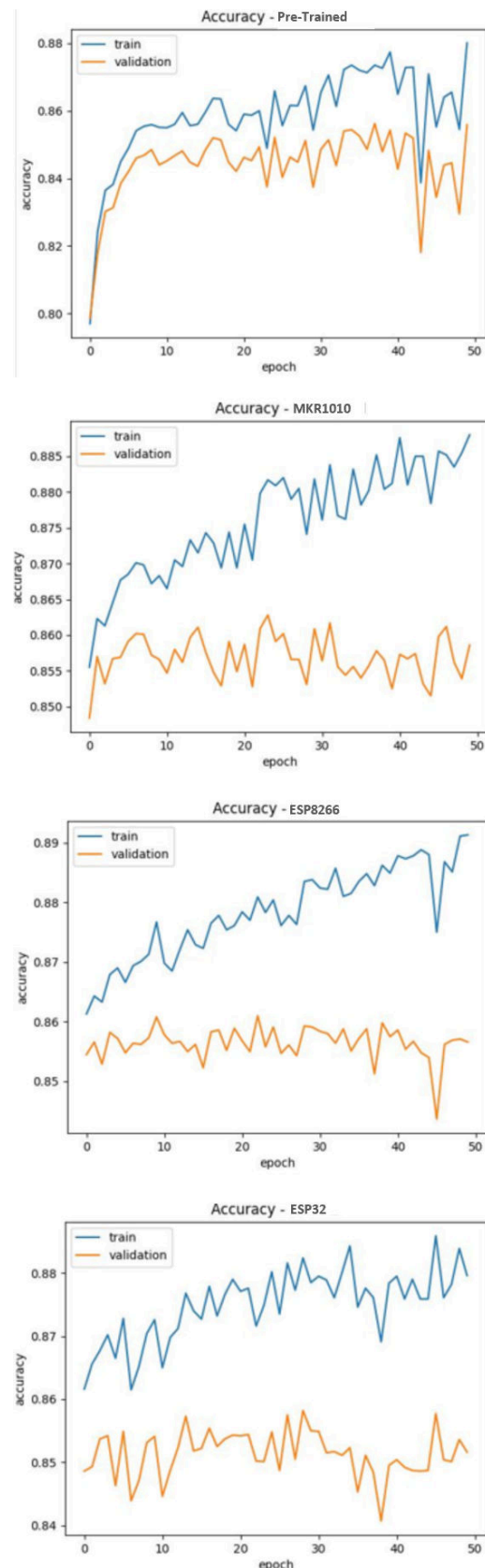


Fig. 2. Accuracy of Federated Learning with Transfer Learning.

TABLE III
FEDERATED LEARNING WITH TRANSFER LEARNING.

Metric	Pre-trained		MCR1010		ESP8266		ESP32		FL
	Training	Testing	Training	Testing	Training	Testing	Training	Testing	
Precision	78.57%	72.21%	81.21%	76.34%	84.12%	77.55%	84.15%	75.89%	77.41%
Accuracy	88.37%	85.32%	89.01%	85.86%	87.25%	85.16%	88.94%	85.66%	86.48%
F1	0.762	0.6996	0.767	0.702	0.741	0.6363	0.772	0.6800	0.704
MCE	0.161	0.171	0.098	0.104	0.077	0.0765	0.074	0.0848	0.071
MSE	0.035	0.038	0.018	0.022	0.016	0.06	0.012	0.015	0.012

TABLE IV
FEDERATED LEARNING WITH UNBALANCED DATASET.

Metric	MKR1010 - Dataset #1		ESP8266 - Dataset #2		ESP32 - Dataset #3		Federated
	Training	Testing	Training	Testing	Training	Testing	
Accuracy	84.01%	73.43%	84.98%	73.01%	85.76%	74.61%	63.01%
MSE	0.051	0.189	0.018	0.157	0.039	0.168	0.257

that the file exists on IPFS and can be used to retrieve the document. Such an identifier is stored within the blockchain.

The overall architecture is shown in Fig. 1. Hyperledger Fabric is used to implement the blockchain [27]. We designed authentication and access control mechanisms to retrieve the IPFS CID and allow local nodes to obtain the optimal model weights to perform the rollback. Specifically, the blockchain runs a smart contract where the legitimate IoT nodes and central nodes have registered themselves by passing a unique identifier and their public key during an enrollment phase that we will describe below. IPFS will store all the model weights in a transparent and immutable way thanks to the CID, which also acts as a checksum. Once these identifiers have been published, it is possible to uniquely identify the parties and perform both global updates and model rollbacks. We will consider the entire FL system to be run by n IoT nodes and a single aggregator. The IoT devices securely hold a secret key SK_i associated with the respective public key PK_i , where $i \in \{1, n\}$, while the central node holds a secret key SK_c and associated public key PK_c .

A. Enrollment

During the enrollment phase, we assume that the smart contract manages the aggregator identity PK_c , which, in our scenario, is represented by the Portenta X8. On the contrary, each IoT device must undergo registration on the blockchain for future interactions. This process begins by evaluating the public key of IoT devices, denoted as PK_i . The aggregator initiates this operation by dispatching a nonce to the devices, encrypted with their respective public keys. If a device successfully decrypts the message, the aggregator can be sure that the specific IoT device holds this specific PK_i . At this point, the aggregator evaluates which is the topic to be assigned to the IoT device, based on internal evaluation. Once decided which topic to assign to the device,

then register this association on the blockchain creating a mapping between $(PK_c(PK_i), topic)$. This can be intended as an Access Control Rule (ACL) for the MQTT system. Notice that to enhance the security of the system we decided to encrypt the PK_i with the public key of the aggregator. This encrypted key serves as a unique identifier for IoT nodes, facilitating model rollback operations.

B. Weights Aggregation

Each device generates local weights W_i^t at time t and sends them to the aggregator. Federated Averaging (FedAvg) is the most commonly used algorithm for aggregating model updates from clients. It has been applied in various domains, such as healthcare and finance. FedAvg can be summarized as follows:

- 1) The aggregator initializes a global model.
- 2) The manager randomly selects a subset of participants based on the available devices or resources for the training process.
- 3) The central server aggregates the weights W_i^t received at time t by applying averaging techniques. All the weights are then summed and divided by the number of participants.
- 4) The updated weights are then sent back to the client devices.
- 5) Steps 2, 3, and 4 are repeated for several rounds till the ends of the epochs or a predefined stop criterion is reached.

The aggregated weights are then sent to IPFS and assigned a unique identifier CID_W^t representing the file in the distributed file system at time t . Subsequently, the aggregator transmits CID_W^t to the smart contract responsible for registering the weights along with additional information. This information includes the current estimated accuracy level, determined as the average of accuracy degrees sent by the local nodes and the timestamp of the current model. It's important to note that

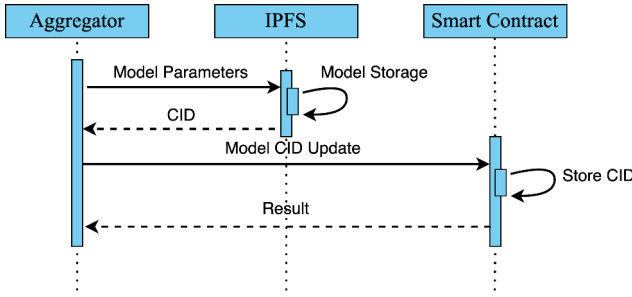


Fig. 3. Model saving on IPFS and Blockchain.

the aggregator may transmit compromised models since no validation occurs during aggregation; however, the timestamp of the current CID_W is recorded in the blockchain and can be leveraged by devices to recover previously not poisoned models.

C. Model Rollback

The concept of a compromised model was introduced in Sec. V-B, and here we delve into the model rollback mechanism used to restore a previous version of the model not yet attacked, also depicted in Alg. 1. To perform this operation the aggregator device establishes a time window of size w , starting at time t . During this time the aggregator is able to aggregate weights, transmit weights to the blockchain, and transmit updated models to the IoT devices. Suppose a local device receives model weights W_k^m from the aggregator at time k with $t \leq k \leq t + w$. At this point, the local device could behave maliciously and start to poison the model. After some prefixed period, the local device transmits the poisoned model to the aggregator. Additionally, the aggregator receives various updated models from the same IoT device at times $(k, k_1, \dots, k_m) \in (t, t + w)$ for the assumption done in defining the time window. During this time the aggregator starts to register the model on the IPFS following the description given in Sec. V-B and Fig. 3. Once the time threshold w is reached, the IoT device initiates a query to the blockchain to retrieve the currently available model. Whenever the aggregator queries the blockchain, an automated process is triggered to evaluate the current accuracy of the model. This assessment involves comparing the current accuracy with the accuracy recorded at the initiation of the time window, denoted as t . If the current accuracy is found to be lower, indicating a potential poisoning attack, the smart contract automatically identifies this and responds to the aggregator with a CID corresponding to a previous version of the model. Conversely, if no attack is detected, the smart contract responds with the CID of the most recent model stored in the IPFS.

D. Weights Propagation

To redistribute the weights, the aggregator must authenticate the local devices. This operation can be done by performing an asymmetric authentication. Fig. 4 depicts the steps needed for exchanging the current model weights W^t . This implementation involves a kind of TLS handshake, without session key

Algorithm 1: Model Rollback.

Input: Time window size w , current time t

Output: Model to be retrieved from IPFS

$t \leftarrow$ current time; $t_{end} \leftarrow t + w$;

while $t_i < t_{end}$ **do**

 Receive CID and average accuracy of updated model from aggregator;

 Insert timestamp and perform transaction;

$t_i \leftarrow$ current time;

end

if accuracy of model at time $t_{end} <$ accuracy of model at time t **then**

 Respond with CID of the model at time t

end

else

 Respond with CID of the model at time t_{end} stored in IPFS;

end

creation. In the first step, the IoT node asks the aggregator to register in a given MQTT topic to be updated on new available models. This message contains the PK_i of a given IoT node. As soon as the aggregator receives this message, then checks by using the smart contract, the presence of the current public key stored in the blockchain during the enrollment phase. If a malicious node tries to access an unauthorized topic, then it will be refused by blocking the entire authentication procedure. If PK_i exists on the blockchain, then the smart contract gives as a response the topic in which the IoT node has been enrolled. After ensuring the authorization, the aggregator creates a new nonce a and sends an encrypted message $m_1 = \text{encrypt}(a, PK_i)$. This nonce is needed in order to avoid possible replay attacks from malicious nodes. If the node is the original one, then it will hold a SK_i associated with the public key existing on the blockchain. At this point, the IoT node is able to decrypt the message and create a new one by including a new nonce and encrypting the previous nonce a and the newer one b with the PK_c held by the aggregator. The aggregator can decrypt this claim using the registered public key, verify the nonce against the last used one, and confirm if there is a match between the registered identifier and the retrieved one. By using the received nonce b , it will send the model W^t to the IoT node, completing the weight propagation procedure. A slightly different procedure involves the creation of a symmetric key derived from a secret session key. We preferred to show this implementation due to the limitation of the IoT node in creating a fresh key each time. Security of the proposed protocol strongly rely on the secrecy of the private key associated with the public one used by the protocol. The use of challenges prevents parties, as well as attackers, from replicating messages. Despite the missing usage of session keys, the protocol is strongly secured by the smart contract containing the authorized public keys of IoT nodes.

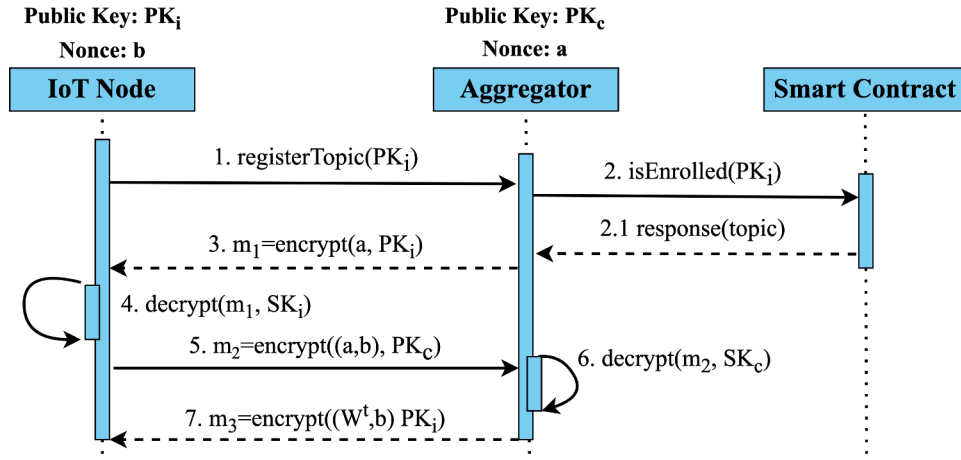


Fig. 4. Authentication between IoT node and Aggregator using PK_i stored on smart contract

VI. CONCLUSIONS AND FUTURE WORK

The TinyML represents an enabler technology in the context of critical IoT applications already integrated with our daily lives, such as transportation, healthcare, and industrial automation. On the other hand, sending data collected on-field to a central server may be limited by legal or privacy constraints. Although still an open challenge, the on-device training on MCUs can support the privacy of data. Moreover, the use of Federated Learning and Transfer Learning represents a good method to improve the accuracy of the ML model executed in resource-constrained MCU-based devices.

To further enhance the performance of this Federated Learning approach, it may be useful to explore the usage of a technology that can improve the communication efficiency between the locally trained model with the aggregator, because some nodes may face limitations due to low network bandwidth or lack of energy, and that, may cause that the global model may become more biased towards the data distribution of those frequently selected clients. To limit the possibility that the latter event happens, there exist compression techniques for the local models may be investigated and applied. It would be useful to explore the effects of weight quantization with a flexible model weight quantization communication scheme that allows the client and the aggregator to flexibly choose the number of bits used to communicate the weights of the model itself. Another efficient compression technique to investigate is the pruning of the model. Pruning allows the user to eliminate all those parameters that give a negligible contribution to the performance of the model itself. Pruning requires criteria for choosing which elements to prune. Both compression techniques can be adopted in the on-line training Federated Learning context. Besides the improvement mentioned above, those techniques would produce also an enhancement concerning the training time itself, because the local model would be able to perform quicker at each round. Finally, a Blockchain-based rollback mechanism can be a valid solution to strengthen the security of the model against possible tampering by maligns.

In the future, we aim to conduct extended experiments of the proposed solution in real-world IoT tasks that involve resource-constrained tiny devices. We will study a deterministically optimal balance of local and global parameters to obtain better model accuracy. Moreover, we aim to introduce mechanisms to identify compromised clients to be excluded from the federated model inference. Formal verification of the issuing and verification protocol is on the roadmap of our research in order to assess the necessity of including session keys. Finally, the used consensus algorithm are not efficient and scalable for MCU-based federated learning, so we will investigate different consensus approaches, such as the Proof-of-Authority, or sharding-based consensus, which are promising solutions for efficiency and scalability in blockchain. Lastly, a human-in-the-loop solution will be designed and implemented in the proposed solution, so that, a human expert can influence the learning process by deciding the local weights to be aggregated, as well as if the global model needs to be rolled back or if the learning should be continued.

ACKNOWLEDGMENT

The work is part of the research activities realized within the projects Federated Learning for Generative Emulation of Advanced Persistent Threats (FLEGREA - CUP E53D23007950001), Bando PRIN 2022, as well as SERICS (CUP PE00000014) under the NRRP MUR program funded by the EU - NGEU.

REFERENCES

- [1] P. Fusco, G.P. Rimoli, and M. Ficco, TinyIDS - An IoT Intrusion Detection System by Tiny Machine Learning, in *Proc. of the 24th Int. Conf. on Computational Science and Its Applications*, 2024.
- [2] Qinyin Li, Zeyi Wen, Zhaomin Wu, Sixu Hu, Naibo Wang, Xu Liu and Bingsheng He, A Survey on Federated Learning Systems: Vision, Hype, and Reality for Data Privacy and Protection, in *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, Apr. 2023, pp. 3347-3366.
- [3] Zheyi Chen, Pu Tian, Weixian Liao, and Wei Yu. Towards multi-party targeted model poisoning attacks against federated learning systems, in *High-Confidence Computing*, vol. 1, no. 1, Jun. 2021.
- [4] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov. How to backdoor federated learning. in *Proc. of the Twenty-Third Int. Conf. on Artificial Intelligence and Statistics*, 2020, pp. 2938–2948.

- [5] R. N. Bhagoji, S. Chakraborty, P. Mittal, and S. Calo. Analyzing federated learning through an adversarial lens. in *Proc. of the Int. Conf. on Machine Learning*, 2019, pp. 634–643.
- [6] D. Dave, S. Parikh, R. Patel, and N. Doshi, A survey on blockchain technology and its proposed solutions,” in *Procedia Computer Science*, vol. 160, 2019, pp. 740–745.
- [7] X. Li, P. Jiang, T. Chen, X. Luo, and Q. Wen, A survey on the security of blockchain systems, in *Future Generation Computer Systems*, vol. 107, 2020, pp. 841–853.
- [8] G.-T. Nguyen and K. Kim, A survey about consensus algorithms used in blockchain, in *Journal of Information processing systems*, vol. 14, no. 1, 2018, pp. 101–128.
- [9] S. K. Lo, Q. Lu, C. Wang, H.-Y. Paik, and L. Zhu. A systematic literature review on federated machine learning: From a software engineering perspective, in *ACM Computing Surveys (CSUR)*, Vol. 54, no. 5, 2021, pp. 1–39.
- [10] C. R. Banbury, V. J. Reddi, M. Lam, and et al. Benchmarking tinyml systems: Challenges and direction, in *arXiv:2003.04821*, 2020.
- [11] R. David, J. Duke, A. Jain, and et al. Tensorflow lite micro: Embedded machine learning on tinyml systems, in *Proc. of the 4th MLSys Conference*, 2021.
- [12] T. Flores, M. Silva, P. Andrade, J. Silva, I. Silva, E. Sisinni, P. Ferrari, and S. Rinaldi. A TinyML Soft-Sensor for the Internet of Intelligent Vehicles, in *Proc. of the IEEE Int. Workshop on Metrology for Automotive (MetroAutomotive)*, 2022, pp. 18–23.
- [13] M. Hashir, N. Khalid, N. Mahmood, M. A. Rehman, M. Asad, M. Q. Mehmood, M. Zubair, and Y. Massoud. 2023. A TinyML based Portable, Low-Cost Microwave Head Imaging System for Brain Stroke Detection, in *Proc. of the IEEE Int. Symposium on Circuits and Systems (ISCAS)*, 2023, pp. 1–4.
- [14] L. Ravaglia, M. Rusci, D. Nadalini, A. Capotondi, F. Conti, and L. Benini. A tinyml platform for on-device continual learning with quantized latent replays. in *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 11, no. 4, 2021, pp. 789–802.
- [15] J. Lin, W.-M. Chen, Y. Lin, j. cohn, C. Gan, and S. Han. Mcunet: Tiny deep learning on iot devices, in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 11711–11722.
- [16] H. Ren, D. Anicic, T. and Runkler. Tinyol: Tinyml with online-learning on microcontrollers, in *Int. Joint Conf. on Neural Networks (IJCNN)*, 2021, pp. 1–8.
- [17] M. Ficco, A. Guerriero, E. Milite, F. Palmieri, R. Pietrantuono, and S. Russo. Federated learning for IoT devices: Enhancing TinyML with on-board training, in *Information Fusion*, vol. 104, 2024, pp. 1–19.
- [18] Y. Lu, X. Huang, K. Zhang, S. Maharjan, and Y. Zhang, Blockchain empowered asynchronous federated learning for secure data sharing in internet of vehicles, in *IEEE Transactions on Vehicular Technology*, vol. 69, no. 4, pp. 4298–4311, 2020.
- [19] Y. Lu, X. Huang, Y. Dai, S. Maharjan, and Y. Zhang, Blockchain and federated learning for privacy-preserved data sharing in industrial iot, in *IEEE Transactions on Industrial Informatics*, vol. 16, no. 6, pp. 4177–4186, 2020.
- [20] U. Majeed and C. S. Hong, Flchain: Federated learning via mec-enabled blockchain network, in *Proc. of the 20th Asia-Pacific Network Operations and Management Symposium (APNOMS)*, 2019, pp. 1–4.
- [21] Y. Zhao, J. Zhao, L. Jiang, R. Tan, D. Niyato, Z. Li, L. Lyu, and Y. Liu, Privacy-preserving blockchain-based federated learning for iot devices, in *IEEE Internet of Things Journal*, vol. 8, no. 3, 2021, pp. 1817–1829.
- [22] S. R. Pokhrel and J. Choi, Federated learning with blockchain for autonomous vehicles: Analysis and design challenges, in *IEEE Transactions on Communications*, vol. 68, no. 8, 2020 pp. 4734–4746.
- [23] Y. Li, C. Chen, N. Liu, H. Huang, Z. Zheng, and Q. Yan, A blockchain-based decentralized Federated Learning framework with committee consensus, in *IEEE Network*, vol. 35, no. 1, 2021, pp. 234–241.
- [24] Houn D. Kwon, Rui Li, Stylianos I. Venieris, J. Chauhan, Nicholas D. Lane, and Cecilia Mascolo. TinyTrain: Deep Neural Network Training at the Extreme Edge, in *arXiv:2307.09988v1* 2023.
- [25] Numpy package for scientific computing in Python, available at <https://numpy.org/> [Last access: Sep. 2023].
- [26] J. Benet, Ipfs-content addressed, versioned, p2p file system, in *arXiv:1407.3561*, 2014.
- [27] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich et al., Hyperledger fabric: a distributed operating system for permissioned blockchains, in *Proc. of the 13th EuroSys conference*, 2018, pp. 1–15.